

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

Facultad de Ciencias e Ingeniería



Guía de Despliegue

Gestión y Planificación de Traslado de Equipaje - Sistema Tasf.B2B

Código	DP-TASF-B2B-2026
Versión	1.1
Fecha	Junio 2026
Curso	1INF54 PDDS - 26-1 - PUCP
Integrantes	Marcelo Jara Falconi Matias Chavarria Rodrigo Lujan Jorge Vicente

1. Máquina Virtual

Características de la máquina virtual sobre la que está desplegado el sistema:

Proveedor	<i>on-premise</i>
Sistema operativo	<i>Ubuntu 22.04 LTS</i>
CPU	<i>2 vCPU Intel(R) Xeon(R) CPU E5-2630 v4 @ 2.20GHz</i>
Memoria RAM	<i>4 GB</i>
Almacenamiento	<i>40 GB</i>
IP / Dominio	<i>200.16.7.154</i>
Usuario de acceso	<i>1inf54.982.8h</i>

Servicios instalados en la VM

- Docker y Docker Compose
- Nginx (proxy reverso, instalado directamente en el host vía apt)
- Git
- Certbot
- MySQL 8

2. Obtención del Código Fuente

El proyecto se encuentra alojado en GitHub y contiene una carpeta para el backend y otra para el frontend.

Comandos — Clonar el repositorio

```
git clone https://github.com/york55/DP1
```

Estructura general del repositorio:

```
[ nombre-del-repositorio ]/  
├── backend/  
├── frontend/  
└── docker-compose.yml
```

3. Configuración del Backend

Archivo de configuración utilizados en la carpeta backend/:

- Dockerfile — imagen del backend

Dockerfile — Backend (pruebas locales opcionales)

```
FROM maven:3.9-eclipse-temurin-21 AS build

WORKDIR /app

COPY pom.xml .

COPY src ./src

RUN mvn clean package -DskipTests

FROM tomcat:10-jdk21

COPY --from=build /app/target/tasfb2b.war /usr/local/tomcat/webapps/

EXPOSE 8080

CMD ["catalina.sh","run"]
```

4. Configuración del Frontend

Archivo de configuración utilizados en la carpeta frontend/:

- Dockerfile — imagen del frontend

Dockerfile — Frontend (pruebas locales opcionales)

```
GNU nano 7.2 Dockerfile
# Etapa 1: compilar React
FROM node:20 AS build

WORKDIR /app

COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
RUN npm run build
```

```
# Etapa 2: servir con nginx  
FROM nginx:alpine
```

```
COPY --from=build /app/dist /usr/share/nginx/html
```

```
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

Archivo de configuración utilizados en la carpeta frontend/:

- nginx.conf

nginx.conf — Proxy Reverso (pruebas locales opcionales)

```
GNU nano 7.2                                nginx.conf  
server {  
  
    listen 80;  
  
    client_max_body_size 500M;  
  
    root /usr/share/nginx/html;  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ /index.html;  
    }  
  
    location /api/ {  
        proxy_pass http://tasfb2b-backend:8080/tasfb2b/api/;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    }  
}
```

```
location /ws {
    proxy_pass http://tasfb2b-backend:8080/tasfb2b/ws;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
}
```

5. Docker y Docker Compose

El archivo docker-compose.yml se encuentra en la raíz del repositorio y orquesta los servicios de backend y frontend (y otros, como base de datos, si aplica).

docker-compose.yml — Construcción y despliegue con Docker Compose

```
services:

  tasfb2b-mysql:
    image: mysql:8
    container_name: tasfb2b-mysql
    environment:
      MYSQL_DATABASE: tasfb2b
      MYSQL_ROOT_PASSWORD: Abcd.1234#
    ports:
      - "3306:3306"
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5

  backend:
    build:
      context: ./backend
    container_name: tasfb2b-backend
    depends_on:
      tasfb2b-mysql:
        condition: service_healthy
    ports:
      - "8080:8080"

  frontend:
    build:
      context: ./frontend
```

```
container_name: tasfb2b-frontend
depends_on:
  - backend
ports:
  - "3000:80"
```

Comandos — Reinicio y limpieza

```
# Descargar los últimos cambios del repositorio
git pull origin main

# Construir las imágenes
docker compose build

# Levantar los contenedores en segundo plano
docker compose up -d

# Ver logs en tiempo real
docker compose logs -f

# Ver estado de los contenedores
docker compose ps

# Reiniciar todos los servicios
docker compose restart

# Detener y eliminar contenedores
docker compose down

# Reconstruir sin usar caché
docker compose build --no-cache

# Eliminar imágenes/contenedores no utilizados
docker system prune -f
```

6. Nginx — Proxy Reverso

Nginx está instalado directamente en la máquina virtual (no dentro de un contenedor) y actúa como proxy reverso, redirigiendo el tráfico hacia los contenedores de backend y frontend según la ruta o el dominio.

Archivo de configuración

- Enlace simbólico en: `/etc/nginx/sites-enabled/default`

```
server {
    listen 80;
    server_name 1inf54-982-8h.inf.pucp.edu.pe;

    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name 1inf54-982-8h.inf.pucp.edu.pe;

    ssl_certificate /etc/letsencrypt/live/1inf54-982-8h.inf.pucp.edu.pe/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/1inf54-982-8h.inf.pucp.edu.pe/privkey.pem;

    client_max_body_size 100M;

    location / {
        proxy_pass http://127.0.0.1:3000; # frontend docker
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8080/tasfb2b/api/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /ws {
        proxy_pass http://127.0.0.1:8080/tasfb2b/ws;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}
```

7. Flujo de Despliegue Resumido

Pasos habituales para desplegar una nueva versión del sistema:

Comandos — Despliegue end-to-end

```
# 1. Conectarse a la VM
ssh 1inf54.982.8h@1inf54-982-8h.inf.pucp.edu.pe
Password: Fs1bDGi0

# 2. Ir al directorio del proyecto
cd /opt/DP1/

# 3. Traer los últimos cambios
sudo git pull

# 4. Reconstruir y levantar los contenedores
docker compose up --build

# 5. Verificar que los contenedores que estén corriendo
docker compose ps

# 6. Remover los contenedores
docker compose down -v

# 6. Verificar que Nginx enrute correctamente
sudo nginx -t && sudo systemctl reload nginx
```

8. Solución de Problemas Comunes

- Puerto en uso: verificar con `sudo netstat -tlnp`
- Contenedor no levanta: revisar `docker compose logs <nombre-contenedor>`
- Si la base de datos ya se había levantado: `sudo systemctl status mysql.service`